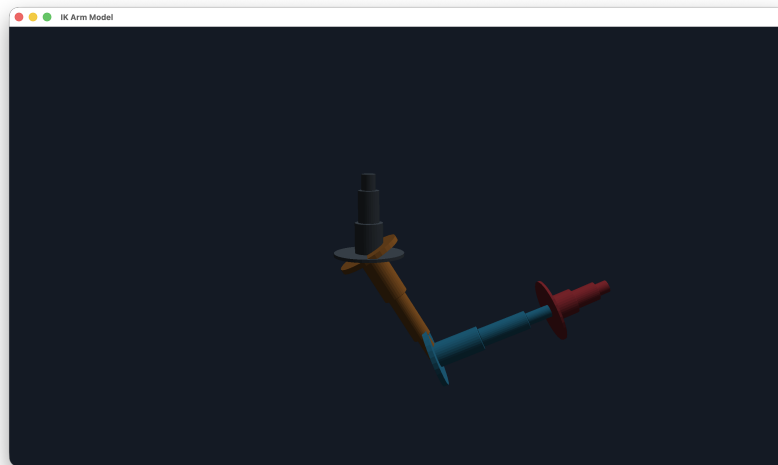


CS7GV5 Assignment 2 Report

Name:	Diksha Chottani
Student ID:	25370268
Youtube link:	https://youtu.be/RVKVrDHk6ec

Required feature: Multi-bone IK in 3D

Screenshot(s) of feature:



Describe your implementation:

The Forward And Backward Reaching Inverse Kinematics (FABRIK) algorithm was implemented to handle a 3D chain consisting of three bones. The solver first checks if the target is within the total reachable length; if not, it aligns the joints in a straight line toward the target. For reachable targets, it performs an iterative process involving a backward pass to the target and a forward pass to the root.

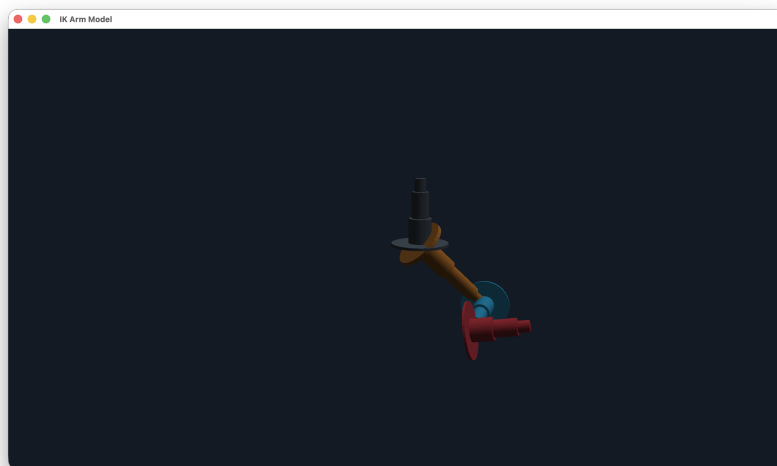
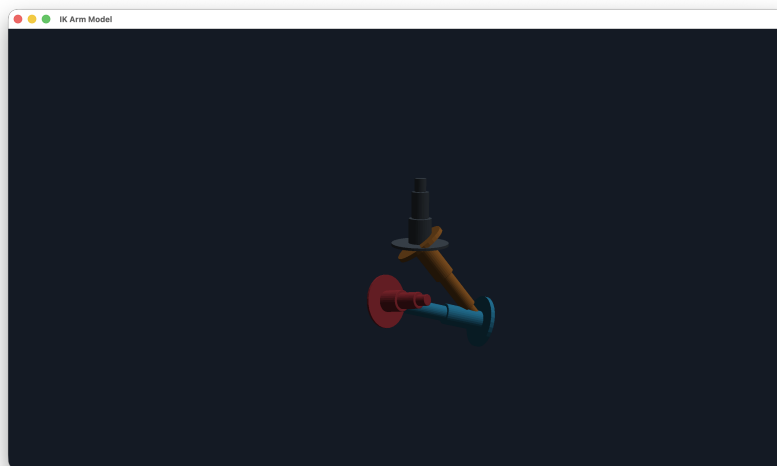
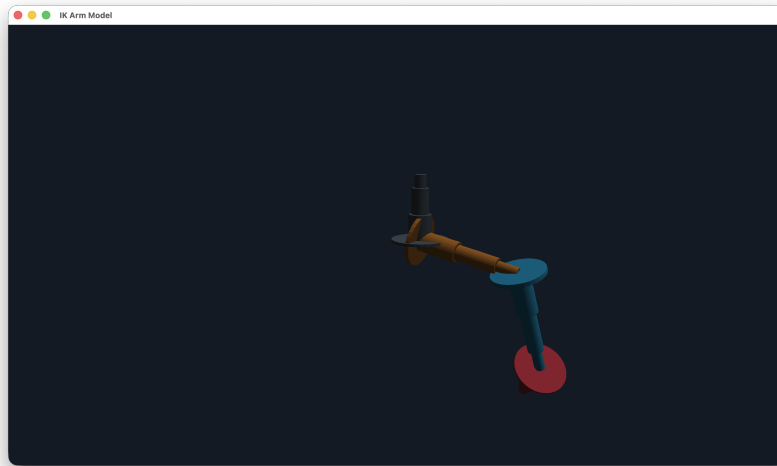
```

void solveFABRIK(const glm::vec3& target, int maxIters=15, float tol=1e-3f){
    float dist = glm::distance(rootStart, target);
    if (dist >= totalLength) {
        for(size_t i=0;i<boneLengths.size();++i){
            float r = glm::distance(joints[i], target);
            float lambda = boneLengths[i] / r;
            joints[i+1] = (1.0f - lambda) * joints[i] + lambda * target;
        }
        return;
    }
    for (int iter=0; iter<maxIters; ++iter){
        joints.back() = target;
        for (int i=(int)joints.size()-2; i>=0; --i){
            glm::vec3 dir = glm::normalize(joints[i] - joints[i+1]);
            joints[i] = joints[i+1] + dir * boneLengths[i];
        }
        joints[0] = rootStart;
        for (size_t i=1; i<joints.size(); ++i){
            glm::vec3 dir = glm::normalize(joints[i] - joints[i-1]);
            joints[i] = joints[i-1] + dir * boneLengths[i-1];
        }
        if (glm::distance(joints.back(), target) < tol) break;
    }
}

```

Required feature: Scripted animation

Screenshot(s) of feature:



Describe your implementation:

A scripted animation using a Catmull-Rom spline was implemented to move the target across four 3D control points. The animation utilizes an `easeInOut` function to smooth the interpolation between points, preventing abrupt velocity changes. The `splineTarget` function calculates the current position on the closed-loop path.

```

static float easeInOut(float t){
    return t*t*(3.0f - 2.0f*t);
}

static glm::vec3 splineTarget(float t){
    std::vector<glm::vec3> P = {
        { 1.2f, 1.6f, 0.4f}, {-0.8f, 1.2f, 0.8f},
        { 0.6f, 1.4f, -0.9f}, { 1.3f, 1.0f, 0.0f}
    };
    float u = fmodf(t, 4.0f);
    int i = (int)floorf(u);
    float local = easeInOut(u - i);
    glm::vec3 p0 = P[(i+3)%4];
    glm::vec3 p1 = P[i%4];
    glm::vec3 p2 = P[(i+1)%4];
    glm::vec3 p3 = P[(i+2)%4];
    return catmullRom(p0,p1,p2,p3, local);
}

```

Required feature: Interactive Mouse-Click Target

Screenshot(s) of feature:



Describe your implementation:

User interactivity is enabled via mouse input, allowing the target to be positioned dynamically. The implementation handles cursor position callbacks to update the camera view and toggle interaction modes, such as enabling or disabling mouse control for precise target placement.

```
static void mouse_callback(GLFWwindow* window, double xposIn, double yposIn) {  
    if (!mouseEnabled) return;  
    float xoffset = static_cast<float>(xposIn) - lastX;  
    float yoffset = lastY - static_cast<float>(yposIn);  
    lastX = static_cast<float>(xposIn);  
    lastY = static_cast<float>(yposIn);  
    camera.ProcessMouseMovement(xoffset, yoffset);  
}
```